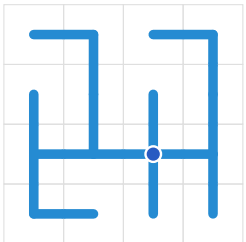
Pipes

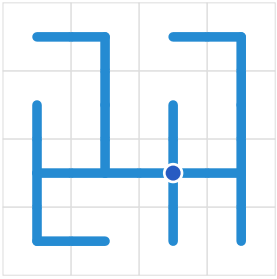
Search for a connected network

Introduction to Artificial Intelligence

Depth-first search and Greedy Best-First



Goal: one connected, acyclic network



- Every port matches its neighbor.
- No port leaves the grid.
- All tiles connect. No cycles.

Tiles are vertices, connections are edges.

$$G \text{ is a tree} \implies |E| = n - 1$$

The blue source anchors water visualization.

Rules: puzzle-pipes.com. Goal validation: model.py and search.py.

Problem formulation

Input	$n = m^2$ tiles, fixed shapes, ordinary boundaries
State	Prefix $s = (a_0, \ldots, a_{k-1})$ of assigned orientations
Initial state	Empty prefix (), with all tile shapes known
Action	Assign a rotation to tile k in row-major order
Transition	$s' = s + (a_k)$, accepted after feasibility checks
Goal	$k = n$ and the complete board passes all rules
Cost	Unit assignment cost, unused by either ranking

A solver assignment can represent several manual clicks.
Any valid solution is acceptable. Minimum-click solutions are outside scope.

Implemented state model: Search in pipes_problem/search.py.

Legal successors

The next tile has a fixed shape. Search chooses its orientation.



NE
reject



ES
keep



SW
reject



NW
reject

Top-left corner example. The left and top borders forbid N and W.

Neighbor matching also checks absent ports.
A candidate must pass every shared pruning check before search accepts it.

Example is schematic. Orientation filtering: boundary_domains() and compatible().

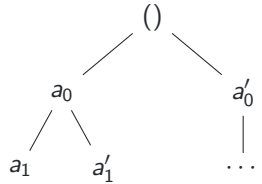
Search space

Distinct rotations per tile

End, elbow, T-junction	4
Straight	2
Cross	1

$$U = \prod_{i=0}^{n-1} |R_i| \leq 4^n$$

Prefix tree



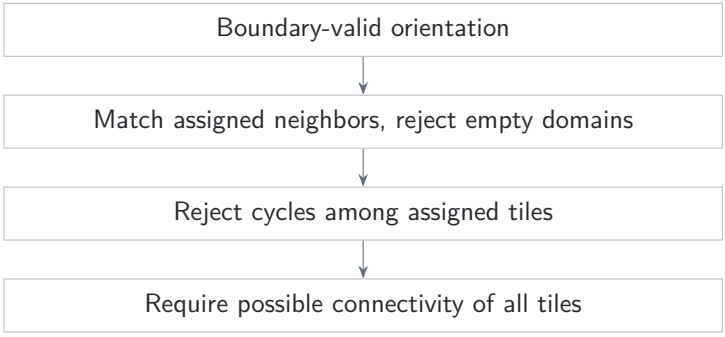
Depth n, branching factor at most 4.
Each prefix has one parent.

Complete assignments before boundary checks.

For the shipped 4×4 example: $U = 268{,}435{,}456$.
This bound counts assignments, not measured expansions.

Count derived from distinct rotations in pipes-4x4-261.json.

Shared pruning



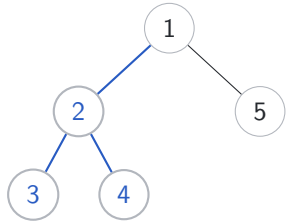
Both solvers use identical checks and fixed tile order.

Root condition: total ports $= 2(n-1)$.
Possible connectivity is necessary but does not prove a solution exists.

Pruning implementation: Search.compatible(). Full reasoning: docs/pipes-analysis.md.

Depth-first search

```
VISIT(s):
  if budget exhausted: return LIMIT
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):
      r = VISIT(child)
      if r is SOLVED or LIMIT: return r
  return UNSOLVABLE
```



Schematic visit order.
Explore a branch, then backtrack.

Finite-tree completeness with sufficient resources and sound pruning.
Worst-case exploration remains exponential.

Simplified pseudocode matches dfs(). Root feasibility precedes VISIT. Cancellation is external.

Greedy's heuristic: remaining choices

$$h(s) = \sum_{i \text{ unassigned}} \log_2 |D_i(s)|$$

$D_i(s)$ contains boundary-valid orientations matching assigned neighbors.

Candidate	Remaining domain sizes	$h(s)$
A	$(1, 2, 4)$	$0 + 1 + 2 = 3$
B	$(1, 1, 2)$	$0 + 0 + 1 = 1$

Greedy prefers B. Smaller values indicate fewer unresolved choices.
Tie order: deeper prefix, then earlier insertion.

The score ignores dependencies between unassigned tiles.
It provides no admissibility or minimum-click guarantee.

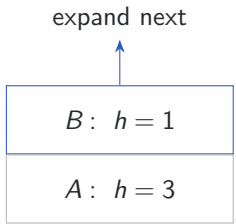
Hypothetical domain example. Exact priority: (h, -depth, insertion order) in greedy().

Greedy Best-First search

```
push () into min-priority queue
while queue is not empty:
  if budget exhausted: return LIMIT
  s = pop minimum-priority state
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    D = FEASIBLE(child)
    if D exists:
      push child with (h(child),
                       -|child|, serial)
return UNSOLVABLE
```

Root feasibility precedes the queue. Priority uses h alone. The unique root starts at zero.

Queue selection



All feasible prefixes remain queued.

Memory can grow exponentially.

Evaluation protocol

- Same nine saved maps for both algorithms.
- Three maps per size, five repeats per algorithm.
- Shared pruning, tile order and resource limits.

Saved map → Timed headless solve → Separate memory replay

Time excludes drawing and playback. Peak memory counts traced Python allocations. Combine passes only when outcomes agree.

Limits: 30 seconds of solver computation and 200,000 expansions.

Runner: benchmark.py. Measurement: measure() and worker.py.

Baseline results

Size	Algorithm	Solved/runs	Time (ms)	Peak (KiB)
4×4	DFS	15/15	0.685	26.984
	Greedy	15/15	0.776	12.359
5×5	DFS	15/15	1.518	53.297
	Greedy	15/15	1.692	19.219
7×7	DFS	15/15	6.603	145.852
	Greedy	15/15	8.213	27.594

Median over 15 successful runs per size and algorithm.
Each group contains three distinct inputs with five timing repeats.

These inputs require little search under shared pruning. Queue overhead can outweigh the benefit of Greedy's ranking. This small sample supports no general winner.

Measured 22 October 2026. Raw rows and environment: docs/benchmarks/.

Scope and next experiments

- Compare harder maps and proven-unsolvable inputs.
- Measure the effect of shared pruning separately.
- Keep UNSOLVABLE distinct from LIMIT or CANCELLED.
- Complete the second puzzle and final course report.

References

Pipes rules: `https://www.puzzle-pipes.com/`
Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Chapters 3 and 6. `https://aima.cs.berkeley.edu/`

Implementation and derived analysis: this repository's `search.py`, `model.py` and `docs/pipes-analysis.md`.